

L17 · Δυναμικός Προγραμματισμός IV

Ανεξάρτητο Σύνολο σε Δέντρα & Bellman-Ford

Algorithms Class Hub

Αλγόριθμοι και Πολυπλοκότητα (K17) · ΕΚΠΑ

Δυναμικός προγραμματισμός σε γραφήματα: ανεξάρτητο σύνολο μέγιστου βάρους σε δέντρα, και ο αλγόριθμος Bellman-Ford για συντομότερες διαδρομές με αρνητικά βάρη.

1 Τι θα δούμε

Η τελευταία διάλεξη του μαθήματος εφαρμόζει τον δυναμικό προγραμματισμό σε **γραφήματα**:

1. **Ανεξάρτητο σύνολο σε δέντρα** — DP όπου τα υποπροβλήματα είναι **υποδέντρα**.
2. **Bellman-Ford** — συντομότερες διαδρομές με **αρνητικά** βάρη, εκεί που ο Dijkstra (από το μάθημα των γραφημάτων) αποτυγχάνει. Κλείνει έτσι μια υπόσχεση που είχαμε αφήσει ανοιχτή.

2 Ανεξάρτητο Σύνολο σε Δέντρα — «το πάρτι»

2.1 Το πρόβλημα

Οργανώνουμε ένα πάρτι για μια εταιρεία με **αυστηρή ιεραρχική δομή**. Κάθε υπάλληλος v έχει μια θετική «προσφορά» $\chi(v)$ (πόσο κέφι φέρνει). Ένας περιορισμός: **δεν** καλούμε ταυτόχρονα έναν υπάλληλο και τον **άμεσο προϊστάμενό** του.

Είσοδος: ένα **δέντρο** που περιγράφει την ιεραρχία, με προσφορά σε κάθε κορυφή.

Στόχος: λίστα καλεσμένων με το **μέγιστο άθροισμα προσφορών**, που να σέβεται τον περιορισμό.

Διαίσθηση

Σε ορολογία γραφημάτων: ζητάμε ένα **ανεξάρτητο σύνολο** (σύνολο κορυφών χωρίς ακμή μεταξύ τους) **μέγιστου βάρους** σε ένα δέντρο. Στα γενικά γραφήματα αυτό το πρόβλημα είναι NP-δύσκολο — αλλά σε **δέντρα** ο δυναμικός προγραμματισμός το λύνει εύκολα.

2.2 Δύο τιμές ανά κορυφή

Το κλειδί: για κάθε κορυφή v , τα υποπροβλήματα αφορούν το **υποδέντρο** της v . Αλλά μία τιμή δεν αρκεί — η απάντηση εξαρτάται από το **αν η ίδια η v είναι μέσα**. Άρα κρατάμε **δύο**:

Κλειδί

- $A[v]$ = μέγιστη προσφορά ενός έγκυρου υποσυνόλου του **υποδέντρου** της v (η v επιτρέπεται ή όχι).

- $B[v]$ = μέγιστη προσφορά ενός έγκυρου υποσυνόλου του υποδέντρου της v που **δεν περιέχει** την v .

Έστω v με άμεσους υφιστάμενους (παιδιά) $v_1, v_2, \dots, v_k$.

Για το $B[v]$: η v **δεν** μπαίνει — άρα δεν μπλοκάρει κανέναν. Κάθε υφιστάμενος μπορεί ελεύθερα να μπει ή όχι· για κάθε υποδέντρο παίρνουμε το βέλτιστο $A[v_i]$:

$$B[v] = \sum_{i=1}^k A[v_i]$$

Για το $A[v]$: δύο σενάρια, παίρνουμε το καλύτερο. Η v **δεν** μπαίνει: αυτό είναι ακριβώς το $B[v]$. Η v **μπαίνει**: τότε **κανένα** από τα παιδιά $v_1, \dots, v_k$ δεν επιτρέπεται, με συνεισφορά $\chi(v) + \sum_i B[v_i]$.

$$A[v] = \max \left\{ B[v], \chi(v) + \sum_{i=1}^k B[v_i] \right\}$$

Σημείωση

Παράδειγμα. Σε ένα μικρό δέντρο με ρίζα r ($\chi = 8$) και παιδιά a ($\chi = 10$) και b ($\chi = 20$), όπου το a έχει παιδιά c ($\chi = 15$) και d ($\chi = 4$), η βέλτιστη λίστα είναι $\{b, c, d\}$ με άθροισμα $20 + 15 + 4 = 39$. Πράγματι $A[r] = \max\{B[r] = 39, \chi(r) + B[a] + B[b] = 8 + 19 + 9 = 36\} = 39$.

2.3 Ο αλγόριθμος

Υπολογίζουμε τα $A[v], B[v]$ **από τα φύλλα προς τη ρίζα** (κάθε κορυφή χρειάζεται πρώτα τα παιδιά της — μια αντίστροφη τοπολογική διάταξη του δέντρου). Για **φύλλο** ℓ : $B[\ell] = 0$ και $A[\ell] = \chi(\ell)$. Η απάντηση είναι το $A[\text{ρίζα}]$.

Κάθε κορυφή και κάθε ακμή επεξεργάζεται $O(1)$ φορές $\rightarrow O(n)$ συνολικά.

3 Συντομότερες Διαδρομές με Αρνητικά Βάρη

3.1 Γιατί ξαναπιάνουμε το πρόβλημα

Στο μάθημα των γραφημάτων ο Dijkstra έλυσε τις συντομότερες διαδρομές — αλλά με τη ρητή **απαίτηση όλα τα βάρη να είναι θετικά**. Τώρα επιτρέπουμε **αρνητικά** βάρη ℓ_e (π.χ. δρομολόγηση όπου κάποιες ακμές δίνουν «κέρδος»).

Προσοχή

Ο Dijkstra μπορεί να αποτύχει. Μόλις «οριστικοποιήσει» μια κορυφή, δεν την ξανακοιτάζει. Μια αρνητική ακμή που οδηγεί σε **ήδη οριστικοποιημένη** κορυφή θα την βελτίωνε — αλλά είναι πια αργά. (Παράδειγμα: με ακμές $s \rightarrow t$ βάρους 2, $s \rightarrow u$ βάρους 1 και $t \rightarrow u$ βάρους -5 , ο Dijkstra οριστικοποιεί το u στο 1 μέσω $s \rightarrow u$ και χάνει το $s \rightarrow t \rightarrow u = 2 + (-5) = -3$.)

3.2 Πρόσθεσε σταθερά; Όχι.

Δελεαστική ιδέα: «πρόσθεσε μια μεγάλη σταθερά σε **όλα** τα βάρη ώστε να γίνουν θετικά, και τρέξε Dijkstra». **Δεν δουλεύει:** μια διαδρομή με πολλές ακμές κερδίζει τη σταθερά **πολλές φορές**, ενώ μια διαδρομή με λίγες ακμές λίγες — η σύγκριση αλλοιώνεται. Ο Dijkstra και πάλι αποτυγχάνει.

3.3 Αρνητικοί κύκλοι

Κλειδί

Αρνητικός κύκλος: ένας κύκλος του οποίου το άθροισμα βαρών είναι **αρνητικό**.

Αν υπάρχει διαδρομή από την s στην t που περνά μέσα από αρνητικό κύκλο, **δεν υπάρχει** συντομότερη s - t διαδρομή — γυρνώντας στον κύκλο μειώνεις το κόστος επ' άπειρον. Διαφορετικά, η συντομότερη διαδρομή **υπάρχει** και είναι **απλή** (χωρίς επαναλαμβανόμενες κορυφές).

4 Ο αλγόριθμος Bellman-Ford

4.1 Η αναδρομή — παραμετροποίηση με το πλήθος ακμών

Σταθεροποιούμε έναν **προορισμό** t . Το έξυπνο υποπρόβλημα:

Κλειδί

$\text{OPT}(i, v)$ = το μήκος του συντομότερου $v \rightarrow t$ μονοπατιού που χρησιμοποιεί **το πολύ** i ακμές.

Έστω P το βέλτιστο τέτοιο μονοπάτι. Δύο περιπτώσεις:

- Το P χρησιμοποιεί **το πολύ** $i - 1$ ακμές — τότε $\text{OPT}(i, v) = \text{OPT}(i - 1, v)$.
- Το P χρησιμοποιεί **ακριβώς** i ακμές. Έστω (v, w) η πρώτη του ακμή· τότε το υπόλοιπο είναι το συντομότερο $w \rightarrow t$ μονοπάτι με $i - 1$ ακμές. Κόστος: $\ell_{vw} + \text{OPT}(i - 1, w)$.

Κλειδί

$$\text{OPT}(i, v) = \begin{cases} 0 & i = 0, v = t \\ \infty & i = 0, v \neq t \\ \min \left\{ \text{OPT}(i - 1, v), \min_{(v,w) \in A} (\ell_{vw} + \text{OPT}(i - 1, w)) \right\} & i > 0 \end{cases}$$

Διαίσθηση

Πόσο μεγάλο πρέπει να γίνει το i ; Αν δεν υπάρχουν αρνητικοί κύκλοι, η συντομότερη διαδρομή είναι **απλή** — δεν επαναλαμβάνει κορυφή — άρα έχει **το πολύ** $n - 1$ ακμές. Συνεπώς το $M[n - 1, v]$ δίνει τη συντομότερη $v \rightarrow t$ απόσταση για **κάθε** κορυφή v .

4.2 Ο πίνακας

```
ΣυντομότεροΜονοπάτι(G, t):  
  Για κάθε  $v \in V$ :  $M[0, v] \leftarrow \infty$   
   $M[0, t] \leftarrow 0$   
  Για  $i = 1$  έως  $n-1$ :  
    Για κάθε  $v \in V$ :  
       $M[i, v] \leftarrow M[i-1, v]$   
      Για κάθε ακμή  $(v, w) \in A$ :  
         $M[i, v] \leftarrow \min(M[i, v], \ell(v, w) + M[i-1, w])$   
  επίστρεψε  $M[n-1, .]$ 
```

Πολυπλοκότητα: $\Theta(mn)$ χρόνος, $\Theta(n^2)$ χώρος.

4.3 Βελτιώσεις

Κλειδί

- **Γραμμικός χώρος.** Κράτα έναν **μονοδιάστατο** πίνακα $M[v] =$ το μήκος του καλύτερου $v \rightarrow t$ μονοπατιού που βρέθηκε ως τώρα. Μετά από i γύρους, το $M[v]$ δεν είναι μεγαλύτερο από το συντομότερο μονοπάτι με $\leq i$ ακμές. Χώρος: $O(m + n)$.
- **Παράλειψη.** Μην εξετάζεις ακμή (v, w) εκτός αν το $M[w]$ **άλλαξε** στον προηγούμενο γύρο — αλλιώς δεν θα δώσει τίποτα νέο.
- **Πρόωρος τερματισμός.** Αν σε έναν ολόκληρο γύρο **καμία** τιμή δεν άλλαξε, σταμάτα — έχεις συγκλίνει.

Με αυτές: $O(m + n)$ χώρος, $O(mn)$ χρόνος στη χειρότερη περίπτωση — αλλά συχνά πολύ ταχύτερα.

Σημείωση

Παρατηρήσεις-ερωτήματα. Ο αλγόριθμος, δοσμένου ενός t , υπολογίζει το συντομότερο $v \rightarrow t$ μονοπάτι για **κάθε** v . Μπορεί να προσαρμοστεί για $s \rightarrow v$ (αντιστρέφοντας τις ακμές), για **DAG** (όπου αρκεί ένα πέρασμα κατά τοπολογική διάταξη), και — αλλάζοντας $\min$ σε $\max$ — για τα **μακρύτερα** μονοπάτια.

Ανακεφαλαίωση

Τι κρατάμε από αυτή τη διάλεξη

1. **Ανεξάρτητο σύνολο σε δέντρα** — DP με υποπροβλήματα τα υποδέντρα. Δύο τιμές ανά κορυφή: $A[v]$ (η v ελεύθερη) και $B[v]$ (η v έξω). $B[v] = \sum A[v_i]$, $A[v] = \max\{B[v], \chi(v) + \sum B[v_i]\}$. $O(n)$.
2. Ο **Dijkstra** **αποτυγχάνει** με αρνητικά βάρη· η προσθήκη σταθεράς **δεν** βοηθάει.
3. **Αρνητικός κύκλος** $\rightarrow$ δεν υπάρχει συντομότερη διαδρομή· αλλιώς υπάρχει και είναι **απλή** ($\leq n - 1$ ακμές).
4. **Bellman-Ford** — $\text{OPT}(i, v) =$ συντομότερο $v \rightarrow t$ μονοπάτι με $\leq i$ ακμές. Αναδρομή: $\min\{\text{OPT}(i-1, v), \min_{(v,w)}(\ell_{vw} + \text{OPT}(i-1, w))\}$. Απάντηση στο $M[n-1, v]$.
5. $\Theta(mn)$ χρόνος· με μονοδιάστατο πίνακα, παράλειψη αμετάβλητων κορυφών και πρόωρο τερματισμό $\rightarrow O(m + n)$ χώρος.

